system design notes Development Handbook

Architecture usage troubleshooting and continuing development

OchsSoft | 26 October 2026

This handbook explains the project from everyday use to the code and data that implement it. Use it to set up a development checkout, follow a request or document through the system, locate the right extension point, make a change, and verify the result.

Begin with the project overview and architecture. The later chapters provide exact local commands, user journeys, worked development examples, diagnostics, recovery steps, known gaps, and a pinned source reference. Technical terms are explained where they are used.

The documented source is COchs-Personal/system-design-notes at d3f6b41b2507f8ec835a070ce53743651ecaab95. Source-defined behavior, observed evidence, planned capabilities, and remaining platform work are distinguished. The quick operations guide remains a separate short reference.

Contents

Understand the project

Understand the project....3
Content architecture and conceptual flows....4

Use and develop the project

Local reading and maintenance on NixOS....6
A practical study session....6
Developing better notes....8

Diagnose improve and verify

Gaps troubleshooting and preservation....9
Source reference....10

Understand the project

This repository is a collection of study notes based on the System Design Interview books, volumes 1 and 2. The root `Readme.md` attributes that origin and calls the notes a work in progress. It links 28 numbered topic directories, a published notes site and additional reading. It is not an implemented backend, tutorial application or benchmark suite.

Use it as a map of design questions and tradeoffs. A diagram of a chat service is an explanatory model, not a deployment that can be started. A storage estimate in a chapter depends on that chapter's assumptions, not measurements of your own product. The most valuable skill is learning to connect a requirement to a design decision and then identify what evidence would validate it.

The chapters progress from scaling and estimation through common infrastructure and product examples: rate limiting, consistent hashing, key-value stores, IDs, URL shortening, crawling, notifications, feeds, chat, autocomplete, video, drive-style storage, geospatial services, queues, monitoring, aggregation, reservations, email, object storage, leaderboards, payments, wallets and exchanges. They need not be read strictly in order, but the first three establish useful vocabulary and an interview framework.

There is no canonical project memory key supplied for this study repository. It remains independent of other project architectures. Do not turn hypothetical chapter diagrams into claims about a live application or import private project details into notes merely to make examples feel concrete.

Attribution matters. The repository's educational origin and external links should remain visible. A local copy of study notes is not permission to reproduce entire source books or redistribute all third-party images. This handbook summarizes the repository's structure and maintenance contracts rather than reproducing the chapters wholesale.

Content architecture and conceptual flows

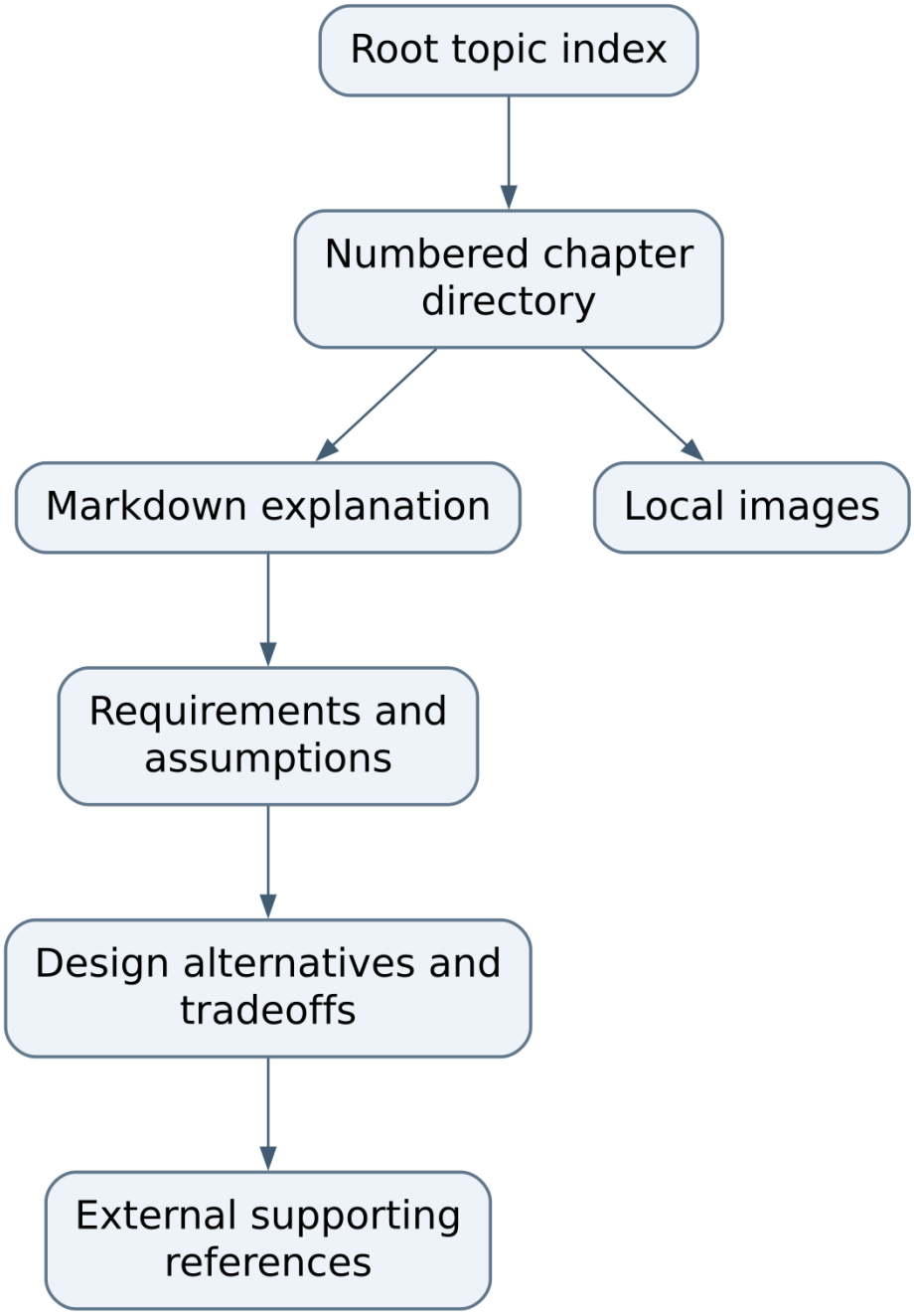


Figure 1 System flow

The source of truth here is text and image files. There is no runtime database, HTTP API, environment variable contract or application state schema. Case and spaces in directory names matter on Linux: early chapters generally use `Readme.md`, later ones often use `README.md`, and the digital-wallet directory contains two spaces after its numeric prefix. Copy actual paths rather than normalizing names casually and breaking links.

Example rate limiter chapter

`04. Rate Limiter/Readme.md` explains placement at the server or middleware, then compares token bucket, leaking bucket, fixed-window counter, sliding-window log and sliding-window counter. Its local images illustrate the models. The distributed discussion names counter synchronization, race conditions and Redis-based coordination. These are alternative design ideas, not an installed Redis service in this repository.

Read an algorithm by asking what state it retains, when it updates that state, what happens at a boundary, and what a rejected request observes. For example, the fixed-window discussion explicitly warns about bursts at adjacent window edges. The next useful maintenance improvement would be a precise worked timeline and assumptions, not a claim that one algorithm is universally best.

Example unique IDs and chat

The unique-ID chapter compares multi-master counters, UUIDs, a ticket server and a Snowflake-style layout. Its example layout uses one sign bit, 41 timestamp bits, five datacenter bits, five machine bits and 12 sequence bits. Those are the chapter's chosen example, not a universal ID standard. Clock rollback and worker identity deserve explicit reasoning before copying such a design into code.

The chat chapter sets a hypothetical scale and covers persistent WebSockets, stateless API work, stateful chat servers, message storage, discovery, notification and presence. It describes per-channel ordering and synchronization using a device's last seen message ID. These concepts explain how the diagram is meant to work; they do not prove delivery guarantees or a working multi-device implementation.

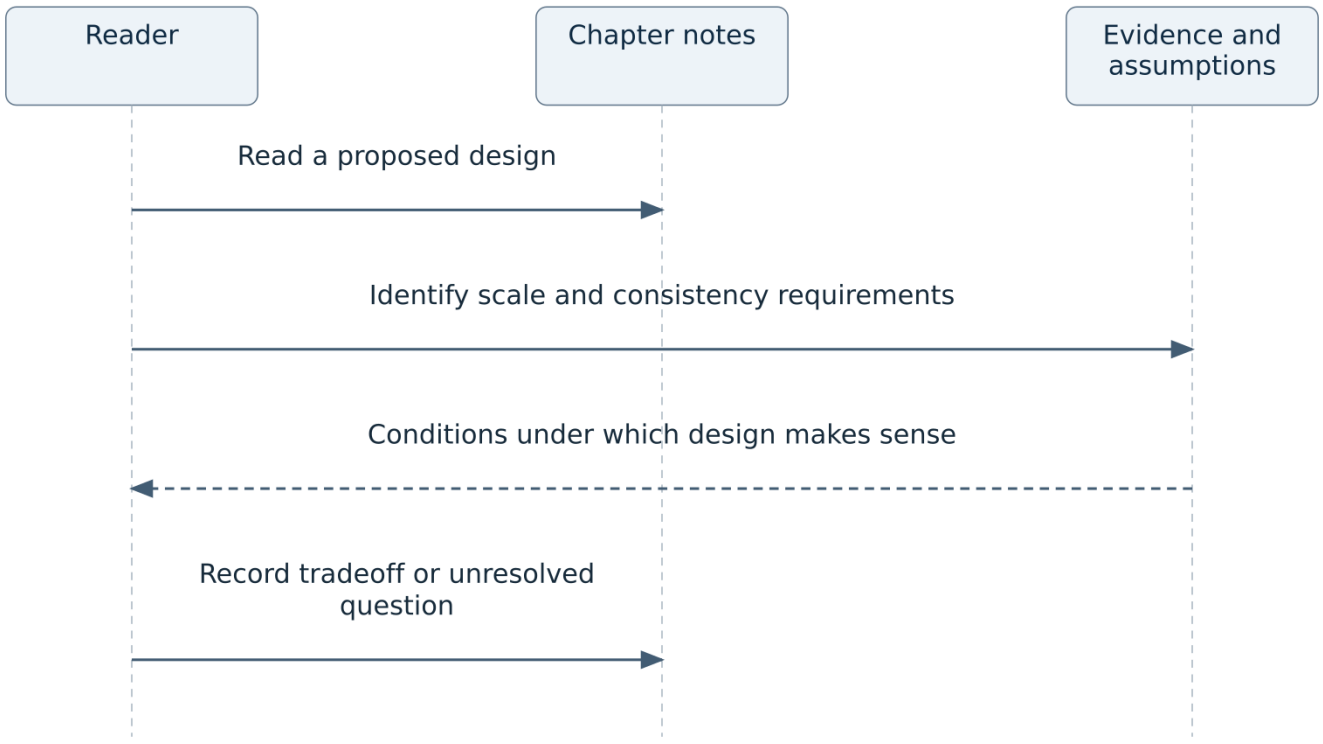


Figure 2 Example: unique IDs and chat

Diagrams and rendering

Many chapters embed HTML `img` elements with width, height or margin styling. GitHub, a Markdown editor and a document exporter may render these differently. The text should explain the diagram's important meaning so an image width or missing asset does not erase the argument. Preserve descriptive alternative text and inspect local paths when a figure disappears.

Local reading and maintenance on NixOS

Git and a Markdown-capable editor are sufficient. There is no dependency manifest, local application server or pinned Nix environment required to read the notes. Open the root index and follow a few representative links. Use exact case-sensitive paths on NixOS. A local renderer may be useful, but choosing one is a tooling decision, not an existing repository command.

```
git status --short --branch
git ls-files
rg -n '<img|!\[|\]\(' Readme.md '04. Rate Limiter' '12. Chat System'
git diff --check
```

The search helps inspect references and image usage. It does not verify that every remote URL still works or that external articles remain current. This handbook describes the committed notes; no remote source verification was performed for its claims. When relying on a linked article for a new technical recommendation, read the primary source and record when it was checked.

Check local links relative to the file containing them. A root link and a chapter's image link have different starting directories. If renaming a chapter, search all incoming references before moving it. Avoid bulk whitespace or capitalization changes mixed with conceptual edits; they make it harder to see whether the technical explanation changed.

No container is required. If future maintenance introduces a renderer in a container, it must use Podman and a reproducible, documented environment. Do not add a complex publishing stack merely to read static notes. Routine hosted CI is disabled under workspace policy; use appropriate local checks and retain human review before publishing.

A documentation test can establish that local files resolve and Markdown structure is sound. It cannot establish that a distributed algorithm tolerates failures. Keep those evidence levels clear when describing validation in a PR.

A practical study session

Start with a question

Choose a concrete question such as “How should a small service reject excessive requests?” Read the rate-limiter requirements before its algorithm catalog. Write down the assumed request volume, burst tolerance, identity key, storage availability and rejection behavior. Then compare the chapter's alternatives against those conditions. Do not choose Redis or a gateway solely because a diagram contains it.

For a chat-design session, identify whether ordering is per conversation or global, what happens when a device reconnects, and whether an acknowledged message is durably stored. The chapter

discusses a synchronization cursor and offline notification. Use those as prompts to ask about missing guarantees instead of assuming the text specifies every retry and duplicate case.

For unique IDs, calculate how many IDs each worker must issue within one time unit, how worker identities are assigned, and what happens when time moves backward. The chapter's bit allocation is a useful example to reason about capacity. A successful reading session can end with an explicit unresolved question; it need not invent a definitive answer unsupported by evidence.

Annotate responsibly

Separate existing notes from your proposed clarification. Add a small worked example or a clearly labeled assumption rather than rewriting an entire chapter around one application's constraints. Preserve source attribution and mark historical product-scale examples as historical examples. If an external link is stale, retain enough title/author context to locate the original source before replacing it with a superficially similar article.

Close and retain useful work

There is no running service to stop. Save deliberate edits, inspect the diff and verify related links and figures. A learning session with no repository change need not produce a commit or operational memory file. If a chapter correction is useful, submit it as a scoped documentation change with the reasoning and source evidence needed by another reader.

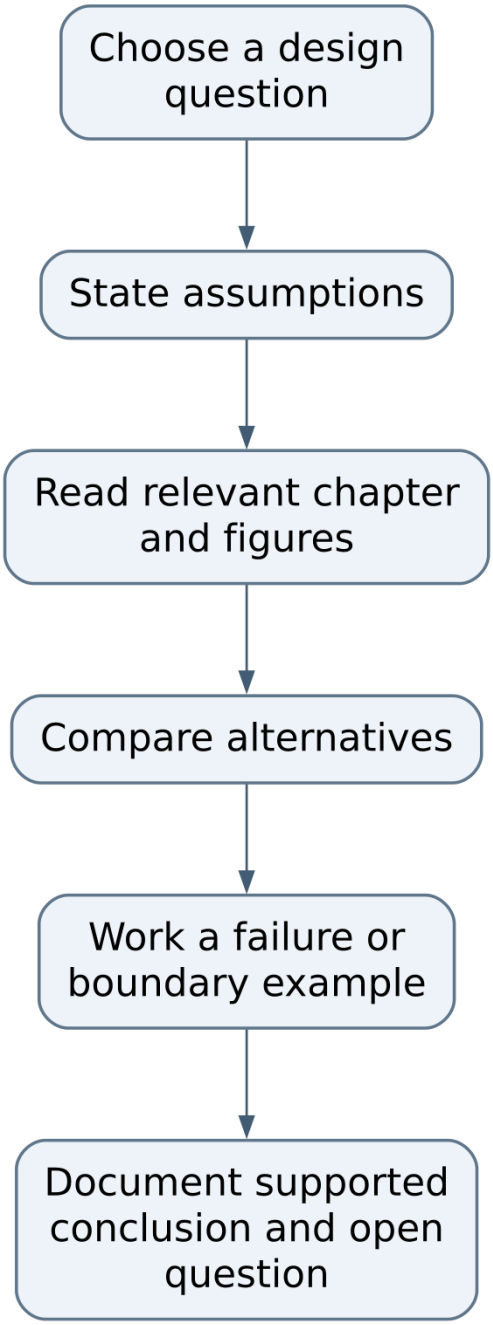


Figure 3 Close and retain useful work

Developing better notes

Recipe clarify a rate limiter boundary case

Open `04. Rate Limiter/Readme.md`, locate the fixed-window explanation and its edge-burst image, and add an original small timeline with a chosen window size and request limit. Show requests just before and just after the boundary and count which are allowed. Explain what property the example demonstrates and which alternative changes it. Keep this as an explanatory example, not a benchmark of a missing implementation.

Review the same language in the image caption and surrounding comparison. Check that “strict,” “approximate,” “burst,” and “rolling” have clear meanings. If proposing an actual implementation later, it belongs in an explicitly scoped code project with concurrency and failure tests, not hidden inside an educational paragraph.

Recipe improve an ID design explanation

In `07. Unique-Id Generator/Readme.md`, connect the example bit allocation to the stated capacity and time horizon. Clarify worker identity and clock behavior with source-backed reasoning. Distinguish uniqueness, numerical order, causality and strict global order: these are different promises. Avoid an unsupported statement that clock synchronization alone solves every rollback or failover case.

Keep worked arithmetic local and reproducible. State units, epoch and assumptions. A reader should be able to recompute the example without needing to trust an unexplained large number. Link to a primary source when adding a technical claim, but do not copy long passages from it.

Recipe repair navigation and images

Find the exact referenced path, including case and spaces. Inspect whether the link is Markdown or HTML and whether it is relative to the current chapter. Repair the smallest broken reference and search for other references to the same file. Open the chapter in the intended renderer and inspect the figure and its alt text. Preserve original images and attribution unless a deliberate replacement is supported and licensed.

When exporting to Word or PDF, check that tables, numbered steps, code examples and image captions remain readable. A successful export command is not visual QA. The editable Markdown should remain the maintainable source, with exports synchronized rather than independently rewritten.

Evidence for a reviewable change

A good PR explains the reader problem, the corrected explanation, source support and local link/render checks. It should not claim tests of a nonexistent deployed system. Keep changes to one concept or navigation problem where possible. Use a topic branch, target the canonical primary branch, and retain human merge authority.

Gaps troubleshooting and preservation

Root chapter link fails

What to inspect. Exact path, capitalization and encoded spaces

Safe repair. Correct the reference to the real directory; search incoming links before renaming

Image missing locally

What to inspect. Chapter-relative path and actual asset file

Safe repair. Restore or correct that asset reference; retain descriptive text

Figure looks different in PDF

What to inspect. HTML styling support and export layout

Safe repair. Adjust supported formatting and visually review the result

Two chapters appear contradictory

What to inspect. Different assumptions, scale or consistency goals

Safe repair. State those assumptions before deciding whether either explanation is wrong

External article disappeared

What to inspect. Original title, author, URL and citation context

Safe repair. Find a legitimate original/archive or mark unavailable; do not silently substitute unrelated advice

Reader expects executable examples

What to inspect. File inventory and chapter scope

Safe repair. Explain that this is a notes collection; propose code work separately

The first improvement priority is a local reference audit across all 28 chapters and their images, including case-sensitive paths. Acceptance is reproducible local link checks plus representative rendering. The second is clearer assumptions and source dates for numerical/product examples. Acceptance is a reader being able to distinguish a worked design from a current operational fact. The third is consistent captions, terminology and accessibility across HTML-heavy and Markdown-heavy chapters.

The root index contains a doubled slash in the rate-limiter directory link and a doubled space in the digital-wallet directory name. These are concrete navigation details to inspect before normalizing anything. A doubled slash may be tolerated by one renderer while failing elsewhere; a directory rename requires updating all references, not merely the displayed title.

Back up the Git repository, including image assets and history. External web pages are not preserved by cloning these notes. An export is useful for reading but cannot replace editable chapter sources and original attribution. If a diagram or note is accidentally removed, compare the known revision in a clean checkout and restore only the intended file while preserving unrelated edits.

No user accounts, database, service ports or live study-session state exist here. Closing the editor is sufficient to end ordinary use. The important recovery criterion is readable, attributable material with working local navigation, not the health of an imagined application.

Source reference

This handbook describes committed source in COchs-Personal/system-design-notes at `d3f6b41b2507f8ec835a070ce53743651ecaab95`. Paths below are relative to this repository. Source-defined behavior is distinguished from a fresh runtime verification; no application or game was launched to author these chapters. The existing quick operating guide remains available separately.

Reading the evidence

A source file explains what the program is designed to do. A test states an expectation and may exercise only one layer. A README describes an intended workflow and can lag implementation. When these disagree, inspect the exact revision and the executable code, reproduce the

disagreement in a safe local test, and correct the relevant documentation with the change. Do not treat a passing unit test as proof of rendering, sound, external connectivity, deployment, or save recovery.

File and symbol index

File	Responsibility	Entry points and locations
Readme.md	Root topic index, attribution and further reading	L1 to L101
04. Rate Limiter/Readme.md	Algorithm comparisons and distributed considerations	L1 to L132
07. Unique-Id Generator/Readme.md	Requirements, alternatives and Snowflake-style example	L1 to L100
12. Chat System/Readme.md	Hypothetical chat requirements, flows, storage and presence	L1 to L204
21. Ad Click Event Aggregation/README.md	Aggregation study chapter and design discussion	L1 to L549

Keeping the handbook useful

When code changes, update the chapter that explains its contract, the relevant diagram, the change recipe, and this pinned source register. Keep the Word and PDF editions synchronized with the Markdown chapters. Record what was actually tested, what remained unverified, and which saved data must survive an upgrade. A future maintainer should be able to trace an instruction to a file or a test without relying on a previous conversation.